3-16. ARTIFICIAL INTELLIGENCE — MODEL ANSWERS

1. The question tests basic understanding of Prolog, data structures, list processing and recursion, and using relations 'backwards' and forwards. It should be relatively straightforward for anyone who and been to the lectures and done all the lab exercises.

   a) A list of atoms, with the end symbol clearly marked e.g.

   ```
   [a,c,end], [a,b,b,b,b,c,end], ... etc.
   ```

   b)
   ```
   startstate( s0 ).
   transition( s0, s1, a ).
   transition( s1, s1, a ).
   transition( s1, s3, c ).
   transition( s1, s2, b ).
   transition( s2, s2, b ).
   transition( s2, s3, c ).
   endstate( s3 ).
   ```

   c)
   ```
   match( RE ) :-
           startstate( S ),
           regexp( RE, S ).

   %with cut
   regexp( [end], S ) :-
           endstate( S ), !.

   match( [H|T], S ) :-
           transition( S, NS, H ),
           regexp( T, NS ).

   %without cut
   regexp( [end], S ) :-
           endstate( S ).

   match( [H|T], S ) :-
           H \= end,
           transition( S, NS, H ),
           regexp( T, NS ).
   ```

   Because a list with just the end of string marker [end] will match with both clauses, in the first case we elimnate the choice point for Prolog on backtracking, in the second Prolog will back track but we use a 'guard' to prevent the second possibility being selected.

   d) Infinite loop or out of local stack, as the first clause of regexp will fail, but RE will unify with [H—T], get the transition to s1, with H=a – and then call regexp(T, s1). But the call of regexp(T, s1) will only result in another call of regexp(T, s1), and so on, forever...

Answers

e)

```
generate1( C, [C] ).

generate_0( _, L, L ).
generate_0( C, L, R ) :-
        generate_0( C, [C|L], R ).

generate_1( C, [C], [C] ).
generate_1( C, [H|T], R ) :-
        generate_1( C, [C,H|T], R ).

gs( L ) :-
        generate_1( a, [a], L1 ),
        generate_0( b, [], L2 ),
        generate1( c, L3 ),
        append( L1, L2, Temp ),
        append( Temp, L3, Lx ),
        append( Lx, [end], L ).
```

The approach breaks down to generating each of the *a*, *b*, and *c* character sequences in turn.

The result is that it generates sequences of ever increasing numbers of zero or more *b*s, between an *a* and a *c*, i.e.

```
[a,c]
[a,b,c]
[a,b,b,c]
[a,b,b,...,b,c]
```

Answers

2. The question was concerned with the the General Graph Search Engine studied in lectures and used in the Prolog labs. Part (a) is an application of state space formulation. Part (b) requires understanding of data representation for GGSE with reference to Part (a) for the exampel. Part (c) is part bookwork but also requires some insight in being able to say why each algorithm is appropriate for this problem or not.

a) State representation: 3-tuple with an atom for which side of the river the boat is on (left, right), list of characters on the left bank, list of characters on the right bank

```
Initial state = (left, [mom,dad,son,son,dau,dau,cop,thf], [] ).

Final state = (right, [], [mom,dad,son,son,dau,dau,cop,thf]).

state_change( boatL2R, (left,LB,RB), (right,NewLB,NewRB) ) :-
        pick2( LB, Move2, NewLB ),
        only_adult( Move2 ),
        append( Move2, RB, NewRB ),
        satisfy_constraints( NewLB ),
        satisfy_constraints( NewRB ).

%%and same for boatR2L rule

only_adult( Move2 ) :-
        member( mom, Move2 ), !.
only_adult( Move2 ) :-
        member( dad, Move2 ), !.
only_adult( Move2 ) :-
        member( policeman, Move2 ).

satisfy_constraints( L ) :-
        dad_daughters_mom( L ),
        mom_sons_dad( L ),
        thief( L ).

dad_daughters_mom( L ) :-
        \+ member( dad, L ), !.
dad_daughters_mom( L ) :-
        member( dad, L ),
        \+ member( daughter, L ), !.
dad_daughters_mom( L ) :-
        member( dad, L ),
        member( daughter, L ), !,
        member( mom, L ).

mom_sons_dad( L ) :-
        \+ member( mom, L ), !.
mom_sons_dad( L ) :-
        member( mom, L ),
        \+ member( son, L ), !.
mom_sons_dad( L ) :-
        member( mom, L ),
        member( son, L ), !,
        member( dad, L ).
```

Answers

```
thief( L ) :-
        \+ member( thief, L ), !.
thief( L ) :-
        member( thief, L ),
        member( policeman, L ), !.
thief( [thief] ).
```

b) state – representation of the problem state
Node = state plus possibly some more information
Path = list of nodes
Graph = list of Paths

Draw out one move of the search space, then show

state = (left, [mom,dad, ...],[])

Node = (RuleUSed, State)

Path list of nodes

Graph list of paths

c) breadth first: expands a node on the search frontier closest to the start node

depth first: expands a node on the search frontier furthest from the start node

uniform cost: expands a node on the search frontier with least cost from start node

best first: expands a node on the search frontier with least estimated cost to a goal node

evaluation criteria: completeness, optimality, time/space complexity wrt to b 'ideal' branching factor and d depth of solution (or m maximum depth of tree)

breadth first: yes, yes (if...), $O(b^d)$ time and space

depth first: no, no, $O(b^d)$ time, $O(b*d)$ space

uniform cost: yes, yes (if...), $O(b^d)$ time and space

best first: no, no, $O(b^m)$ time and space

breadth first: branching factor very high

depth first: watch out for loops

uniform cost: what is the cost

best first: if one can think of a heuristic

Answers

3. This question is in three parts, generally not very deep in each part, but repays lecture attendance and reading the primers, by covering some of the 'marginal' topics, such as 2-player games and modal logic, and the proof of optimatity for A^*.

a) S is the start state, Op is the set of state change operators

$$P_{G'} = \bigcup_{i=0}^{\infty} P'_i$$
$$P'_0 = \{[S]\}$$
$$P'_{i+1} = \{[n \mid p] \mid \exists p \in P'_i . \exists op \in Op . op(frontier(p)) = (n, e)\}$$

Provided: graph G is *rooted* on S (i.e. all paths in G start from S), and:

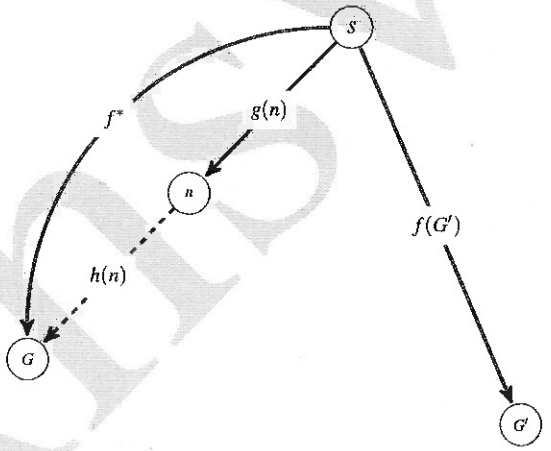
$$(n, e, n') \in R \leftrightarrow \exists op_i \in Op . op_i(n) = (n', e)$$

b) To prove that A* is optimal:

- Let G be an optimal goal state, with path cost f^*
- Let G" be a sub-optimal goal state with path cost $f(G')$. Then

$$f(G') = g(g') + h(G') = g(G') + 0 = g(G')$$

- Suppose A* returns G' as its (sub-optimal) solution, so $f(G') > f^*$, so $g(G') > f^*$.
- Now consider a node n on the frontier node on a path leading to G



$f^* \geq f(n)$	h is admissible and the path cost is monotonically increasing
$f(n) \geq f(G')$	otherwise n would have been selected for expansion before G'
$f^* \geq f(G')$	transitivity
$f^* \geq g(G')$	

- This is a contradiction
- So either G' was optimal, or A* never expands a sub-optimal goal

Answers

c) minimax does exhaustive search and labels end nodes with 0 (win for min) or 1 (win for max)

minimax with fixed ply does exhaustive search to depth d (the ply), and assigns each node a value according to a heuristic function

alpha beta stores an alpha cut-off value with max nodes and a beta cut-off value with min nodes, the do depth first search, evaluate the node, and offer this value to parent and grandparent etc. nodes as possible limit for pruning the search tree

d) Assume that players still have same the same knowledge and making individual attempts to win over the other two

Assign three layers, max, min and other. if a player is eliminated by a move, then if it is max, assign it 0, otherwise switch to rules for 2-player game

Can still try to do exhaustive search, assign nodes 0 if it is a win for min or for other, can still inherit for max best value of child-nodes, for either min or other the worst value (assuming it does not matter which of these is going to win)

Answers

e) $\mathscr{M} = \langle W, R, [\![]\!] \rangle$

where:

- W is a non-empty set, the set of all **possible worlds**
- R is a binary relation on W, $R \subseteq (W \times W)$, the **accessibility relation** between possible worlds
- $[\![]\!]$ is the **denotation** function from propositional symbols to subsets of W (it denotes the possible worlds in which the proposition is true)

The meaning of a *wff* of propositional modal logic is defined by:

$$
\begin{aligned}
\models^{\mathscr{M}}_{\alpha} p \quad & \textit{iff} \quad \alpha \in [\![p]\!] \\
\models^{\mathscr{M}}_{\alpha} \neg p \quad & \textit{iff} \quad \textit{not} \models^{\mathscr{M}}_{\alpha} p \\
\models^{\mathscr{M}}_{\alpha} p \wedge q \quad & \textit{iff} \quad \models^{\mathscr{M}}_{\alpha} p \text{ and } \models^{\mathscr{M}}_{\alpha} q \\
\models^{\mathscr{M}}_{\alpha} p \vee q \quad & \textit{iff} \quad \models^{\mathscr{M}}_{\alpha} p \text{ or } \models^{\mathscr{M}}_{\alpha} q \\
\models^{\mathscr{M}}_{\alpha} p \rightarrow q \quad & \textit{iff} \quad \text{if } \models^{\mathscr{M}}_{\alpha} p \text{ then } \models^{\mathscr{M}}_{\alpha} q \\
\models^{\mathscr{M}}_{\alpha} \Box p \quad & \textit{iff} \quad \forall \beta \in \mathscr{M}. \alpha R \beta \rightarrow \models^{\mathscr{M}}_{\beta} p \\
\models^{\mathscr{M}}_{\alpha} \Diamond p \quad & \textit{iff} \quad \exists \beta \in \mathscr{M}. \alpha R \beta \wedge \models^{\mathscr{M}}_{\beta} p
\end{aligned}
$$

f) (*i*)

1	1 :	$\neg(\Box P \rightarrow P)$	1,negated conclusion
2	1 :	$\Box P$	1,α
3	1 :	$\neg P$	1,α
4	1 :	P	2,$\Box$

close

(*ii*)

1	1 :	$\neg(P \rightarrow \Box\Diamond P)$	1,negated conclusion
2	1 :	P	1,α
3	1 :	$\neg(\Box\Diamond P)$	1,α
4	2 :	$\neg\Diamond P$	3,$\Diamond$
5	1 :	$\neg P$	4, $\Box$

close

(*iii*)

1	1 :	$\neg(\Box P \rightarrow \Box\Box P)$	1,negated conclusion
2	1 :	$\Box P$	1,α
3	1 :	$\neg(\Box\Box P)$	1,α
4	2 :	$\neg\Box P$	3,$\Diamond$
5	3 :	$\neg P$	4, $\Diamond$
6	3 :	P	2, $\Box$

close

These are the characteristic formulas for reflexivity, symmetry and transitivity, S5 is the logic in which the incidence relation satisfies all those properties.

Answers

4. A test of resolution theorem proving ad understanding of first order logic, and a bit of Prolog.

a) Prolog processes queries left to right using matching with clauses top to bottom, with backtracking on fail

Unification is a process of attempting to syntactically identify two symbolic expressions by the matching of terms and the replacement of certain sub-expressions (variables) by other expressions

Resolution is an inference rule, general case:

$$\frac{\begin{array}{l}\neg q_1 \vee \neg q_2 \vee \ldots \vee \neg q_m \vee p_i \\ \neg p_1 \vee \neg p_2 \vee \ldots \vee \neg p_{i-1} \vee \neg p_i \vee \neg p_{i+1} \vee \ldots \vee \neg p_m\end{array}}{\neg p_1 \vee \neg p_2 \vee \ldots \vee \neg p_{i-1} \vee (\neg q_1 \vee \neg q_2 \vee \ldots \vee \neg q_m) \vee \neg p_{i+1} \vee \ldots \vee \neg p_m}$$

Prolog tries to match the query with the head of a clause with unification, then applies resolution to replace the head in the query with the body of the clause

b) say yes

says no

Prolog is sound, it is just not complete.

c) (i) says false

(ii) says true

(iii) says false

Using the closed-world assumption, anything not provably true is actually false. So in the third case I cannot prove there is something to unify with X who likesCats, but in the second case I cannot prove that jeremy likeDogs, so false, but "not" (slash-plus) of false is true.

d) $\forall x.\forall y.hunter(x) \rightarrow insecure(y)$
$\forall x.\forall y.lion(x) \rightarrow threat(y)$
$\forall x.\forall y.insecure(x) \wedge threat(y) \rightarrow hunts(x,y)$
$\forall x.\forall y.hunts(x,y) \wedge shoots(x,y) \rightarrow kills(x,y)$
$hunter(walter)$
$lion(cecil)$
$shoots(walter,cecil)$

$\neg hunter(x1) \vee insecure(x1)$
$\neg lion(x2) \vee threat(x2)$
$\neg insecure(x3) \vee \neg threat(y3) \vee hunts(x3,y3)$
$\neg hunts(x4,y4) \vee \neg shoots(x4,y4) \vee kills(x4,y4)$
$hunter(walter)$
$lion(cecil)$
$shoots(walter,cecil)$

Answers

e) $\neg kills(walter, cecil)$
$\neg hunts(x4, y4) \vee \neg shoots(x4, y4)\ x4 = walter, y4 = cecil$
$\neg hunts(x4, y4)$
$\neg insecure(x3) \vee \neg threat(y3)\ x3 = x4 = walter, y3 = y4 = cecil$
$\neg hunter(x1) \vee \neg threat(y3)\ x1 = x3 = x4 = walter, y3 = cecil$
$\neg threat(y3)\ y3 = y4 = cecil$
$\neg lion(x2)\ x2 = y3 = y4 = cecil$
$\perp$

Answers

5. The question is designed to test the full range of applications of the Calculus KE for propositional logic, in proof, disproof, and model building.

a) for $S \vdash P$, take distributed-and over premises in S, use deduction theorem to prove $\vdash S' \rightarrow P$, so list premises and negate conclusion, then expand tree according to calculus rules, if get all branches closed then proof formulas is valid.

Sound and complete, can specify rules to control proof which do not affect completeness (e.g. order, only applying rules once, beta-simplification etc.)

b)
```
?- ke( [s->p, w& -z, -p, -z->(s+q+r), (w+y)-> -q, -r], [], [], [], 0 ).
beta  -s
alpha w
alpha -z
beta  s+q+r
beta  q+r
beta  q
beta  - (w+y)
alpha -w
alpha -y
close -w,w
```

c)
```
?- ke( [s->p, w& -z, -z->(s+q+r), (w+y)-> -q, -r], [], [], [], 0 ).
alpha w
alpha -z
beta  s+q+r
     br 1  -s
     beta  q+r
     beta  q
     beta  - (w+y)
     alpha -w
     alpha -y
     close -w,w
     br 2  s
     beta  p
          br 1  - (w+y)
          alpha -w
          alpha -y
          close -w,w
          br 2  w+y
          beta  -q
          open [-q,s,p,-r,-z,w]
```

d)
```
?- ke( [p+ -q + r, -q-> -p, r-> (-p+q), -r], [], [], [], 0 ).
     br 1  - -q
     dne   q
          br 1  p
          open [p,q,-r]
          br 2  -p
          beta  -q+r
          beta  r
          close r,-r
     br 2  -q
     beta  -p
     beta  -q+r
```

Answers

```
        open [-q,-p,-r]
```

e)

```
?- ke( [(p-> -(q+r)), (p&q), (q<->r)], [], [], [], 0 ).
alpha p
alpha q
eta   r
beta  - (q+r)
alpha -q
alpha -r
close -q,q
true.

?- ke( [(p-> -(q+r)), (p&q), -(q<->r)], [], [], [], 0 ).
alpha p
alpha q
eta   -r
beta  - (q+r)
alpha -q
alpha -r
close -q,q
true.

?- ke( [(p-> -(q+r)), -(p&q), (q<->r)], [], [], [], 0 ).
     br 1  q
     eta   r
     beta  -p
     open [-p,q,r]
     br 2  -q
     eta   -r
          br 1  -p
          open [-p,-q,-r]
          br 2  p
          beta  - (q+r)
          alpha -q
          alpha -r
          open [p,-r,-q,-q,-r]
true.

?- ke( [(p-> -(q+r)), -(p&q), -(q<->r)], [], [], [], 0 ).
     br 1  q
     eta   -r
     beta  -p
     open [-p,q,-r]
     br 2  -q
     eta   r
          br 1  -p
          open [-p,-q,r]
          br 2  p
          beta  - (q+r)
          alpha -q
          alpha -r
          close -r,r
true.
```

Answers

```
?- ke( [-(p-> -(q+r)), (p&q), (q<->r)], [], [], [], 0 ).
alpha p
alpha - - (q+r)
dne   q+r
alpha p
alpha q
eta   r
open [r,q,p,p]
true.

?- ke( [-(p-> -(q+r)), (p&q), -(q<->r)], [], [], [], 0 ).
alpha p
alpha - - (q+r)
dne   q+r
alpha p
alpha q
eta   -r
open [-r,q,p,p]
true.

?- ke( [-(p-> -(q+r)), -(p&q), (q<->r)], [], [], [], 0 ).
alpha p
alpha - - (q+r)
dne   q+r
beta  -q
eta   -r
beta  r
close r,-r
true.

?- ke( [-(p-> -(q+r)), -(p&q), -(q<->r)], [], [], [], 0 ).
alpha p
alpha - - (q+r)
dne   q+r
beta  -q
eta   r
open [r,-q,p]
true.
```

So

accept, reject, accept

accept, reject, reject

reject, accept, accept,

reject, accept, reject

reject, reject, reject (just say no)

Answers